

Exp 5 - Write the python program for Missionaries Cannibal problem.

Aim:

To write a python program for Missionaries Cannibal problem.

Algorithm:

- Step 1:** Create a data structure to represent the state of the problem, including the number of missionaries and cannibals on each side of the river and the position of the boat.
- Step 2:** Create a data structure to represent the state space of the problem, including the initial state, the goal state, and the possible transitions between states.
- Step 3:** Use a search algorithm, such as depth-first search or breadth-first search, to explore the state space and find a solution.
- Step 4:** At each step of the search, generate all possible successor states from the current state by applying one of the allowed boat movements (two missionaries, two cannibals, or one of each).
- Step 5:** Check if the successor state is valid (i.e., no more cannibals than missionaries on either side of the river).
- Step 6:** If the successor state is valid, add it to the search tree and continue the search.
- Step 7:** If the successor state is the goal state, return the path from the initial state to the goal state.
- Step 8:** If there are no more possible successor states to explore, backtrack to the previous state and try another possible movement.

Program:

```
from collections import deque
def is_valid_state(state):
    m1, c1, boat, m2, c2 = state
    if m1 < 0 or c1 < 0 or m2 < 0 or c2 < 0:
        return False

    if (m1 > 0 and m1 < c1):
```

```

        return False
    if (m2 > 0 and m2 < c2):
        return False
    return True

def get_successors(state):
    m1, c1, boat, m2, c2 = state
    successors = []
    if boat == 1: # Boat on left side
        moves = [(1,0), (2,0), (0,1), (0,2), (1,1)]
        for m, c in moves:
            new_state = (m1-m, c1-c, 0, m2+m, c2+c)
            if is_valid_state(new_state):
                successors.append(new_state)
    else:
        moves = [(1,0), (2,0), (0,1), (0,2), (1,1)]
        for m, c in moves:
            new_state = (m1+m, c1+c, 1, m2-m, c2-c)

            if is_valid_state(new_state):
                successors.append(new_state)
    return successors

def bfs():
    initial_state = (3, 3, 1, 0, 0)
    goal_state = (0, 0, 0, 3, 3)
    queue = deque([(initial_state, [])])
    visited = set()
    while queue:
        state, path = queue.popleft()
        if state == goal_state:
            return path + [state]

```

```

        if state in visited:
            continue
        visited.add(state)
        for successor in get_successors(state):
            queue.append((successor, path + [state]))
    return
solution = bfs()
print("Solution Path:")
for state in solution:
    print(state)

```

Output:

```

Solution Path:
(3, 3, 1, 0, 0)
(3, 1, 0, 0, 2)
(3, 2, 1, 0, 1)
(3, 0, 0, 0, 3)
(3, 1, 1, 0, 2)
(1, 1, 0, 2, 2)
(2, 2, 1, 1, 1)
(0, 2, 0, 3, 1)
(0, 3, 1, 3, 0)
(0, 1, 0, 3, 2)
(1, 1, 1, 2, 2)
(0, 0, 0, 3, 3)

```

Result:

The program was executed and the output was found to be correct